Technical Report-Assignment 9

2371039 송승희

1. Two-Layer Neural Network (`neural_net.py`)

1.1 개요

이 프로그램은 CIFAR-10 데이터셋을 이용해 Two-Layer Fully-Connected Neural Network을 구현하고 학습시키는 코드이다.

네트워크 구조는 다음과 같다.

- **구조**: input → FC layer → ReLU → FC layer → Softmax
- **가중치**: W_1, W_2와 편향 b_1, b_2
- **예측 출력**: $y' = W_2(\text{ReLU}(W_1 x + b_1)) + b_2$
- **손실 함수**: Softmax Cross-Entropy Loss + L2 Regularization

구현은 `classifier/neural_net.py`의 `TwoLayerNet` 클래스 안에 `loss()`, `train()`, `predict()` 세 함수로 나뉘며, `two_layer_net.ipynb`에서 단계별로 검증하고 학습한다.

1.2 Code Explanation

1.2.1 `__init__()` - 가중치 초기화

```
self.params['W1'] = std * np.random.randn(input_size, hidden_size)
self.params['b1'] = np.zeros(hidden_size)
self.params['W2'] = std * np.random.randn(hidden_size, output_size)
self.params['b2'] = np.zeros(output_size)
```

`std=1e-4`에 비례하는 작은 랜덤값으로 W_1, W_2를 초기화하고, 편향 b_1, b_2는 0으로 초기화한다. 가중치를 너무 크게 초기화하면 ReLU 이후 gradient vanishing 또는 exploding이 발생할 수 있으므로 작은 값으로 시작한다.

1.2.2 `loss()` - Forward Pass

```
s1 = X.dot(W1) + b1    # 1층 선형 변환
p1 = np.maximum(0, s1) # ReLU 활성화
scores = p1.dot(W2) + b2  # 2층 선형 변환
```

입력 X에 대해 1층 선형 변환 후 ReLU를 적용하고, 결과를 2층 선형 변환에 통과시켜 클래스별 점수 `scores`를 계산한다. ReLU는 음수 값을 0으로 만들어 비선형성을 부여한다.

1.2.3 `loss()` - Loss 계산 (Softmax + L2 Regularization)

```
exp_s = np.exp(scores)
p2 = exp_s / np.sum(exp_s, axis=1, keepdims=True)  # Softmax

correct = -np.log(p2[np.arange(N), y])
data_loss = np.sum(correct) / N

regularization = reg * (np.sum(W1*W1) + np.sum(W2*W2))
loss = data_loss + regularization
```

`scores`에 Softmax를 적용해 각 클래스의 확률 `p2`를 구하고, 정답 클래스의 로그 확률의 음수 평균을 데이터 손실로 계산한다. 여기에 W_1, W_2의 제곱합에 비례하는 L2 정규화 손실을 더해 최종 손실을 구한다.

$$L = \frac{1}{N}\sum_i -\log p_{y_i} + \text{reg} \cdot (\sum W_1^2 + \sum W_2^2)$$

1.2.4 `loss()` - Backward Pass (Backpropagation)

```
dscores = p2
dscores[np.arange(N), y] -= 1
dscores /= N

grads['W2'] = p1.T.dot(dscores) + 2 * reg * W2
grads['b2'] = np.sum(dscores, axis=0)

dp1 = dscores.dot(W2.T)
dp1[p1 <= 0] = 0  # ReLU gradient

grads['W1'] = X.T.dot(dp1) + 2 * reg * W1
grads['b1'] = np.sum(dp1, axis=0)
```

연쇄 법칙(chain rule)을 적용해 손실에 대한 각 파라미터의 기울기를 계산한다. Softmax-CrossEntropy의 기울기는 `p2`에서 정답 위치만 1을 빼는 것으로 간단히 표현된다. ReLU의 역전파는 순전파에서 0 이하였던 뉴런의 기울기를 0으로 마스킹한다. L2 정규화 항의 미분인 $2 \cdot \text{reg} \cdot W$를 각 가중치 기울기에 더한다.

1.2.5 `train()` - SGD 학습

```
random_indices = np.random.choice(num_train, batch_size)
X_batch = X[random_indices]
y_batch = y[random_indices]

for i in self.params:
    self.params[i] -= learning_rate * grads[i]

learning_rate *= learning_rate_decay
```

매 iteration마다 훈련 데이터에서 `batch_size`개를 무작위로 샘플링해 미니배치를 구성하고, `loss()`로 기울기를 계산한 뒤 SGD로 파라미터를 업데이트한다. 매 epoch마다 `learning_rate_decay`를 곱해 학습률을 점진적으로 감소시킨다.

1.2.6 `predict()` - 클래스 예측

```
s1 = X.dot(self.params['W1']) + self.params['b1']
p1 = np.maximum(0, s1)
s2 = p1.dot(self.params['W2']) + self.params['b2']
y_pred = np.argmax(s2, axis=1)
```

Forward pass를 수행한 뒤 각 샘플에 대해 클래스 점수가 가장 높은 인덱스를 `argmax`로 선택해 예측 레이블로 반환한다.

1.3 Result Analysis

하이퍼파라미터 탐색은 hidden size `[100, 150]`, learning rate `[1e-3, 3e-3]`, regularization strength `[0.1, 0.5]`의 조합으로 수행하였으며, 각 조합에 대해 1000 iteration 학습 후 validation accuracy를 측정해 가장 높은 모델을 `best_net`으로 저장하였다.

CIFAR 데이터셋을 이용해 구현한 신경망을 테스트한 결과는 아래와 같다. 마크다운과 함께 코드가 설명 되어있다.

---

2. Jupyter Notebook 실행 결과

Implementing a Neural Network

In this exercise we will develop a neural network with fully-connected layers to perform classification, and test it out on the CIFAR-10 dataset.

[1]:
```
# A bit of setup

import numpy as np
import matplotlib.pyplot as plt # library for plotting figures

from classifiers.neural_net import TwoLayerNet

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

We will use the class `TwoLayerNet` in the file `classifiers/neural_net.py` to represent instances of our network. The network parameters are stored in the instance variable `self.params` where keys are string parameter names and values are numpy arrays. Below, we initialize toy data and a toy model that we will use to develop your implementation.

[2]:
```
# Create a small net and some toy data to check your implementations.
# Note that we set the random seed for repeatable experiments.

input_size = 4
hidden_size = 10
num_classes = 3
num_inputs = 5

def init_toy_model():
    np.random.seed(0)
    return TwoLayerNet(input_size, hidden_size, num_classes, std=1e-1)

def init_toy_data():
    np.random.seed(1)
    X = 10 * np.random.randn(num_inputs, input_size)
    y = np.array([0, 1, 2, 2, 1])
    return X, y

net = init_toy_model()
X, y = init_toy_data()
```

Forward pass: compute scores

Open the file `classifiers/neural_net.py` and look at the method `TwoLayerNet.loss`. This function is very similar to the loss functions you have written for the SVM and Softmax exercises: It takes the data and weights and computes the class scores, the loss, and the gradients on the parameters.

Implement the first part of the forward pass which uses the weights and biases to compute the scores for all inputs.

[3]:
```
scores = net.loss(X)
print('Your scores:')
print(scores)
print()
print('correct scores:')
correct_scores = np.asarray([
  [-0.81233741, -1.27654624, -0.70335995],
  [-0.17129677, -1.18803311, -0.47310444],
  [-0.51590475, -1.01354314, -0.8504215 ],
  [-0.15419291, -0.48629638, -0.52901952],
  [-0.00618733, -0.12435261, -0.15226949]])
print(correct_scores)
print()

# The difference should be very small. We get < 1e-7
print('Difference between your scores and correct scores:')
print(np.sum(np.abs(scores - correct_scores)))
```

```
Your scores:
[[-0.81233741 -1.27654624 -0.70335995]
 [-0.17129677 -1.18803311 -0.47310444]
 [-0.51590475 -1.01354314 -0.8504215 ]
 [-0.15419291 -0.48629638 -0.52901952]
 [-0.00618733 -0.12435261 -0.15226949]]

correct scores:
[[-0.81233741 -1.27654624 -0.70335995]
 [-0.17129677 -1.18803311 -0.47310444]
 [-0.51590475 -1.01354314 -0.8504215 ]
 [-0.15419291 -0.48629638 -0.52901952]
 [-0.00618733 -0.12435261 -0.15226949]]

Difference between your scores and correct scores:
3.6802720926321086e-08
```

Forward pass: compute loss

In the same function, implement the second part that computes the data and regularization loss.

[4]:
```
loss, _ = net.loss(X, y, reg=0.05)
correct_loss = 1.30378789133

# should be very small, we get < 1e-12
print('Difference between your loss and correct loss:')
print(np.sum(np.abs(loss - correct_loss)))
```

```
Difference between your loss and correct loss:
1.7985612998927536e-13
```

Backward pass

Implement the rest of the function. This will compute the gradient of the loss with respect to the variables `W1`, `b1`, `W2`, and `b2`. Now that you (hopefully!) have a correctly implemented forward pass, you can debug your backward pass using a numeric gradient check:

```
from gradient_check import eval_numerical_gradient ...
```

```
W2 max relative error: 3.440708e-09
b2 max relative error: 3.865080e-11
W1 max relative error: 3.561318e-09
b1 max relative error: 2.738421e-09
```

Train the network

To train the network we will use stochastic gradient descent (SGD), similar to the SVM and Softmax classifiers. Look at the function `TwoLayerNet.train` and fill in the missing sections to implement the training procedure. This should be very similar to the training procedure you used for the SVM and Softmax classifiers. You will also have to implement `TwoLayerNet.predict`, as the training process periodically performs prediction to keep track of accuracy over time while the network trains.

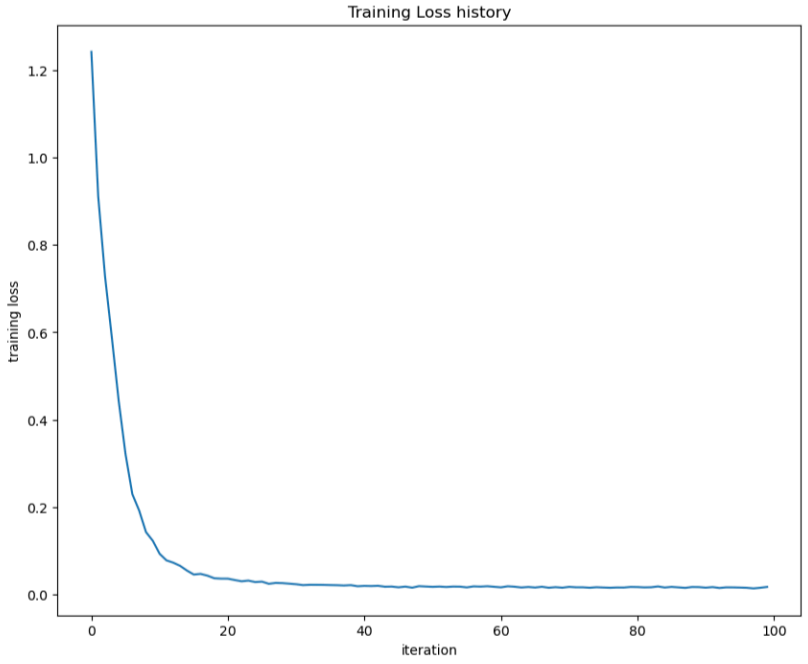
Once you have implemented the method, run the code below to train a two-layer network on toy data. You should achieve a training loss less than 0.02.

[6]:
```
net = init_toy_model()
stats = net.train(X, y, X, y,
            learning_rate=1e-1, reg=5e-6,
            num_iters=100, verbose=False)

print('Final training loss: ', stats['loss_history'][-1])

# plot the loss history
plt.plot(stats['loss_history'])
plt.xlabel('iteration')
plt.ylabel('training loss')
plt.title('Training Loss history')
plt.show()
```

```
Final training loss:  0.01714960793873202
```



Load the data

Now that you have implemented a two-layer network that passes gradient checks and works on toy data, it's time to load up our favorite CIFAR-10 data so we can use it to train a classifier on a real dataset.

[8]:
```
from data_utils import load_CIFAR10

def get_CIFAR10_data(num_training=19000, num_validation=1000, num_test=1000):
    """
    Load the CIFAR-10 dataset from disk and perform preprocessing to prepare
    it for the two-layer neural net classifier. These are the same steps as
    we used for the SVM, but condensed to a single function.
    """
    # Load the raw CIFAR-10 data
    cifar10_dir = 'datasets/cifar-10-batches-py'

    # Cleaning up variables to prevent loading data multiple times (which may cause memory issue)
    try:
       del X_train, y_train
       del X_test, y_test
       print('Clear previously loaded data.')
    except:
       pass

    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    # train / test -> train + val / test
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    # Normalize the data: subtract the mean image
    mean_image = np.mean(X_train, axis=0)
    X_train -= mean_image
    X_val -= mean_image
    X_test -= mean_image

    # Reshape data to rows
    X_train = X_train.reshape(num_training, -1)
    X_val = X_val.reshape(num_validation, -1)
    X_test = X_test.reshape(num_test, -1)

    return X_train, y_train, X_val, y_val, X_test, y_test


# Invoke the above function to get our data.
X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Train data shape:  (19000, 3072)
Train labels shape:  (19000,)
Validation data shape:  (1000, 3072)
Validation labels shape:  (1000,)
Test data shape:  (1000, 3072)
Test labels shape:  (1000,)
```

Train a network

To train our network we will use SGD. In addition, we will adjust the learning rate with an exponential learning rate schedule as optimization proceeds; after each epoch, we will reduce the learning rate by multiplying it by a decay rate.

[9]:
```
input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
net = TwoLayerNet(input_size, hidden_size, num_classes)

# Train the network
stats = net.train(X_train, y_train, X_val, y_val,
            num_iters=1000, batch_size=200,
            learning_rate=1e-4, learning_rate_decay=0.95,
            reg=0.25, verbose=True)

# Predict on the validation set
val_acc = (net.predict(X_val) == y_val).mean()
print('Validation accuracy: ', val_acc)
```

```
iteration 0 / 1000: loss 2.302984
iteration 100 / 1000: loss 2.302664
iteration 200 / 1000: loss 2.299105
iteration 300 / 1000: loss 2.278315
iteration 400 / 1000: loss 2.214040
iteration 500 / 1000: loss 2.135265
iteration 600 / 1000: loss 2.071108
iteration 700 / 1000: loss 2.081200
iteration 800 / 1000: loss 2.075023
iteration 900 / 1000: loss 2.008127
Validation accuracy:  0.241
```

Debug the training

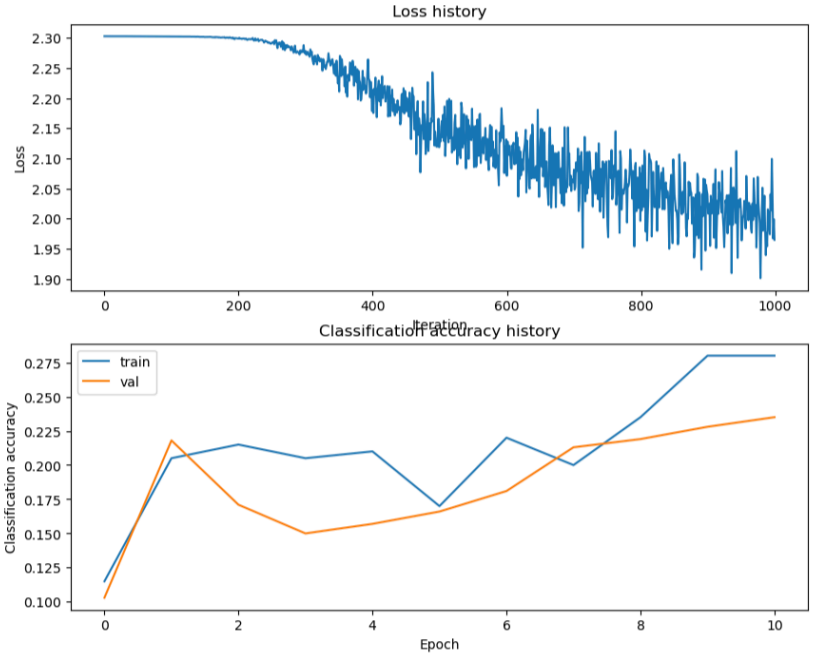
With the default parameters we provided above, you should get a validation accuracy of about 0.27 on the validation set. This isn't very good.

One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

[10]:
```
# Plot the loss function and train / validation accuracies
plt.subplot(2, 1, 1)
plt.plot(stats['loss_history'])
plt.title('Loss history')
plt.xlabel('Iteration')
plt.ylabel('Loss')

plt.subplot(2, 1, 2)
plt.plot(stats['train_acc_history'], label='train')
plt.plot(stats['val_acc_history'], label='val')
plt.title('Classification accuracy history')
plt.xlabel('Epoch')
plt.ylabel('Classification accuracy')
plt.legend()
plt.show()
```



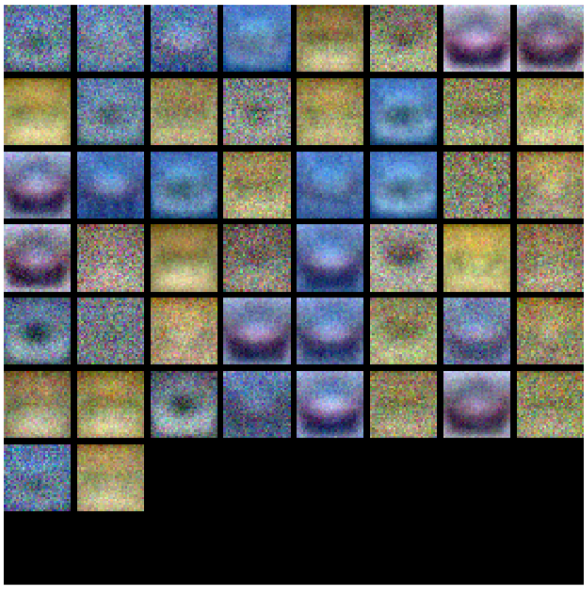
Epoch

[11]:
```
from vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(32, 32, 3, -1).transpose(3, 0, 1, 2)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(net)
```



Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, numer of training epochs, and regularization strength. You might also consider tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 36% on the validation set. Our best network gets over 39% on the validation set.

Experiment: You goal in this exercise is to get as good of a result on CIFAR-10 as you can (39% could serve as a reference), with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

Explain your hyperparameter tuning process below.

Your Answer :

hidden size, learning rate, regularization strength 세 가지 하이퍼파라미터를 조합하여 탐색하였다. hidden size는 [100, 150], learning rate는 [1e-3, 3e-3], regularization strength는 [0.1, 0.5]로 설정하고 모든 조합에 대해 학습을 반복하였다. 각 조합마다 validation accuracy를 측정하여 가장 높은 정확도를 기록한 모델을 best_net으로 저장하였다. learning_rate_decay는 0.95로 고정하여 학습이 진행될수록 학습률이 조금씩 감소하도록 하였다.

[12]:
```
best_net = None # store the best model into this

################################################################################
# TODO: Tune hyperparameters using the validation set. Store your best trained  #
# model in best_net.                                                            #
#                                                                               #
# To help debug your network, it may help to use visualizations similar to the  #
# ones we used above; these visualizations will have significant qualitative    #
# differences from the ones we saw above for the poorly tuned network.          #
#                                                                               #
# Tweaking hyperparameters by hand can be fun, but you might find it useful to  #
# write code to sweep through possible combinations of hyperparameters          #
# automatically like we did on the previous exercises.                          #
################################################################################
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

# Your code

best_val_acc = -np.inf  # 최고 정확도

input_size = 32 * 32 * 3

hidden_sizes = [100, 150]     # 은닉층 증가
learning_rates = [1e-3, 3e-3] # 학습률 변경
regs = [0.1, 0.5]             # L2 정규화
num_classes = 10

for h in hidden_sizes:
    for l in learning_rates:
      for r in regs:
          net = TwoLayerNet(input_size, h, num_classes)

          stats = net.train(X_train, y_train, X_val, y_val,
                            num_iters=1000, batch_size=200,
                            learning_rate=l, learning_rate_decay=0.95,
                            reg=r, verbose=True)

          val_acc = (net.predict(X_val) == y_val).mean()

          if val_acc > best_val_acc:
              best_val_acc = val_acc
              best_net = net

          print(f'Hidden size: {h}, Learning rate: {l}, Regularization strength: {r}, Validation accuracy: {val_acc:.4f}')


# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
```

```
iteration 0 / 1000: loss 2.302921
iteration 100 / 1000: loss 1.916081
iteration 200 / 1000: loss 1.816215
iteration 300 / 1000: loss 1.625303
iteration 400 / 1000: loss 1.690572
iteration 500 / 1000: loss 1.531264
iteration 600 / 1000: loss 1.484228
iteration 700 / 1000: loss 1.470460
iteration 800 / 1000: loss 1.325012
iteration 900 / 1000: loss 1.428285
Hidden size: 100, Learning rate: 0.001, Regularization strength: 0.1, Validation accuracy: 0.4700
iteration 0 / 1000: loss 2.304132
iteration 100 / 1000: loss 2.042462
iteration 200 / 1000: loss 1.801063
iteration 300 / 1000: loss 1.737312
iteration 400 / 1000: loss 1.716282
iteration 500 / 1000: loss 1.578527
iteration 600 / 1000: loss 1.609253
iteration 700 / 1000: loss 1.560403
iteration 800 / 1000: loss 1.448465
iteration 900 / 1000: loss 1.430312
Hidden size: 100, Learning rate: 0.001, Regularization strength: 0.5, Validation accuracy: 0.4390
iteration 0 / 1000: loss 2.302906
iteration 100 / 1000: loss 1.839470
iteration 200 / 1000: loss 1.683523
iteration 300 / 1000: loss 1.694083
iteration 400 / 1000: loss 1.419703
iteration 500 / 1000: loss 1.658967
iteration 600 / 1000: loss 1.563548
iteration 700 / 1000: loss 1.424474
iteration 800 / 1000: loss 1.387946
iteration 900 / 1000: loss 1.263654
Hidden size: 100, Learning rate: 0.003, Regularization strength: 0.1, Validation accuracy: 0.4580
iteration 0 / 1000: loss 2.304115
iteration 100 / 1000: loss 1.832270
iteration 200 / 1000: loss 1.782581
iteration 300 / 1000: loss 1.762216
iteration 400 / 1000: loss 1.574620
iteration 500 / 1000: loss 1.670464
iteration 600 / 1000: loss 1.659263
iteration 700 / 1000: loss 1.648295
iteration 800 / 1000: loss 1.657475
iteration 900 / 1000: loss 1.623916
Hidden size: 100, Learning rate: 0.003, Regularization strength: 0.5, Validation accuracy: 0.4610
iteration 0 / 1000: loss 2.303079
iteration 100 / 1000: loss 1.878371
iteration 200 / 1000: loss 1.790790
iteration 300 / 1000: loss 1.579267
iteration 400 / 1000: loss 1.531515
iteration 500 / 1000: loss 1.523589
iteration 600 / 1000: loss 1.455626
iteration 700 / 1000: loss 1.444698
iteration 800 / 1000: loss 1.472485
iteration 900 / 1000: loss 1.276968
Hidden size: 150, Learning rate: 0.001, Regularization strength: 0.1, Validation accuracy: 0.4470
iteration 0 / 1000: loss 2.304937
iteration 100 / 1000: loss 1.964303
iteration 200 / 1000: loss 1.770505
iteration 300 / 1000: loss 1.733350
iteration 400 / 1000: loss 1.594946
iteration 500 / 1000: loss 1.518571
iteration 600 / 1000: loss 1.632427
iteration 700 / 1000: loss 1.508130
iteration 800 / 1000: loss 1.512266
iteration 900 / 1000: loss 1.552579
Hidden size: 150, Learning rate: 0.001, Regularization strength: 0.5, Validation accuracy: 0.4540
iteration 0 / 1000: loss 2.303029
iteration 100 / 1000: loss 1.696321
iteration 200 / 1000: loss 1.778080
iteration 300 / 1000: loss 1.530756
iteration 400 / 1000: loss 1.498409
iteration 500 / 1000: loss 1.455376
iteration 600 / 1000: loss 1.511160
iteration 700 / 1000: loss 1.365767
iteration 800 / 1000: loss 1.754742
iteration 900 / 1000: loss 1.405648
Hidden size: 150, Learning rate: 0.003, Regularization strength: 0.1, Validation accuracy: 0.4340
iteration 0 / 1000: loss 2.304897
iteration 100 / 1000: loss 1.773266
iteration 200 / 1000: loss 1.787105
iteration 300 / 1000: loss 1.641396
iteration 400 / 1000: loss 1.727872
iteration 500 / 1000: loss 1.742063
iteration 600 / 1000: loss 1.614225
iteration 700 / 1000: loss 1.606183
iteration 800 / 1000: loss 1.506834
iteration 900 / 1000: loss 1.575629
Hidden size: 150, Learning rate: 0.003, Regularization strength: 0.5, Validation accuracy: 0.4460
```

[13]:
```
# visualize the weights of the best network
show_net_weights(best_net)
```

Run on the test set

When you are done experimenting, you should evaluate your final trained network on the test set; you should get above 36%.

[14]:
```
test_acc = (best_net.predict(X_test) == y_test).mean()
print('Test accuracy: ', test_acc)
```

```
Test accuracy:  0.475
```

이미지들 다 확인했어요! 결과 분석 써드릴게요.

2.1 Result Analysis

2.1.1 구현 검증 (Gradient Check)

Forward pass: compute scores에서 목표 오차 1e-7보다 작은 약 3.68e-8, Forward pass: compute loss에서 목표 1e-12보다 작은 약 1.79e-13의 오차가 계산되었으며, Backward pass에서 W2, b2, W1, b1 오차 또한 각각 약 3.44e-9, 3.87e-11, 3.56e-9, 2.74e-9로 목표 오차 1e-8보다 작았다. 따라서 `neural_net.py`의 `loss()`가 적절히 구현된 것을 확인할 수 있다. Train the network에서도 최종 훈련 손실이 약 0.0171로 목표인 0.02보다 작으므로 `train()` 함수 또한 올바르게 동작한다.

2.1.2 초기 학습 결과 (hidden size=50, lr=1e-4)

CIFAR-10 데이터셋을 훈련 19000개, validation 1000개, 테스트 1000개로 분할하여 기본 하이퍼파라미터 (hidden size=50, learning rate=1e-4, reg=0.25)로 1000 iteration 학습한 결과 validation accuracy가 0.241로 낮게 나타났다. Loss history 그래프에서 손실이 2.30에서 약 2.00으로 느리게 감소하고, Classification accuracy history에서 훈련 정확도와 validation 정확도 사이에 큰 차이가 없는 것을 확인할 수 있다. 이를 통해 학습률이 지나치게 낮고 모델 크기 또한 작았기 때문에 이러한 문제가 발생했음을 추론할 수 있다.

2.1.3 하이퍼파라미터 튜닝 결과

이 문제를 해결하기 위해 hidden size `[100, 150]`, learning rate `[1e-3, 3e-3]`, regularization strength `[0.1, 0.5]`의 조합으로 총 8가지 경우를 탐색하였다. 각 조합에 대해 1000 iteration 학습 후 validation accuracy를 측정하였으며 결과는 아래와 같다.

hidden size	learning rate	reg	validation accuracy
100	1e-3	0.1	0.4700
100	1e-3	0.5	0.4390
100	3e-3	0.1	0.4580
100	3e-3	0.5	0.4610
150	1e-3	0.1	0.4470
150	1e-3	0.5	0.4540
150	3e-3	0.1	0.4340
150	3e-3	0.5	0.4460

hidden size=100, learning rate=1e-3, reg=0.1 조합에서 validation accuracy 0.4700으로 가장 높은 성능을 보여 해당 모델을 `best_net`으로 저장하였다.

2.1.4 최종 테스트 결과

`best_net`으로 테스트 데이터셋을 예측한 결과 **Test accuracy: 0.475**로, 목표치인 36% 이상을 크게 상회하였다. `best_net`의 가중치 W_1을 시각화한 결과, 초기 학습 시 노이즈가 많고 불규칙했던 가중치에 비해 튜닝 후에는 더 다양한 색상 패턴과 방향성이 나타나며 의미 있는 특징을 학습했음을 확인할 수 있다.